



操作系统概念 Operating System Concepts

Silberschatz, Galvin and Gagne

王冬青

wang.box@163.com



同濟大學軟件學院

内容框架



- ┆ 概述
- ┆ 体系结构
- ┆ 进程管理
- ┆ 内存管理
- ┆ 文件管理
- ┆ 外设管理



同濟大學軟件學院



内容



- ✚ Ch4: 进程
- ✚ Ch5: 线程
- ✚ Ch6: CPU调度
- ✚ Ch7: 进程同步
- ✚ Ch8: 死锁





Ch 7: Process Synchronization



- Background
- 临界区问题/互斥问题
- 进程同步
- 典型同步互斥问题
- 举例





Ch 7: Process Synchronization



- Background
- 临界区问题/互斥问题
- 进程同步
- 典型同步互斥问题
- 举例



Shared array[100];
Shared index;

Process 1:

n = random();
array[index] = n;

- index++;
(repeat)

Process 2:

m = random();
array[index] = m;

index++;
(repeat)

- What if process 1 loses its quanta before executing index++

– What will happen?

PROCESS2
GETS TO
OVERWRITE
THE VALUE

!



Background



- Concurrent access(并行存取) to shared data may result in data inconsistency. — 数据一致性问题
- Maintaining data consistency requires mechanisms to ensure the **orderly** execution of **cooperating processes**.
- Race condition(竞争条件):
 - ✓ The situation where several processes access and manipulate shared data concurrently.
 - ✓ The final value of the shared data depends upon which process finishes last.
- TO PREVENT RACE CONDITIONS, CONCURRENT PROCESSES MUST BE **SYNCHRONIZED** (同步机制) .



同濟大學軟件學院



Ch 7: Process Synchronization



- Background
- 临界区问题/互斥问题
- 进程同步
- 典型同步互斥问题
- 举例





临界区问题/互斥问题



- 临界区概念
- 临界区使用原则
- 解决互斥问题
 - 软件方法
 - 硬件方法





临界区问题/互斥问题



临界资源

系统中某些资源一次只允许一个任务使用，这样的资源为**临界资源**或**互斥资源**。

如同一进程内各线程对全局变量的访问，或不同进程间的共享互斥资源（打印机、共享文件等）的访问。

互斥：进程内部和进程之间

临界区（互斥区）- Critical section

在进程中涉及到临界资源的程序段叫**临界区**。

剩余区：进程中除临界区外的那部分

同濟大學軟件學院





```
Shared array[100];  
Shared index;
```

Process 1:

```
n = random();
```

```
array[index] = n;  
index++;
```

(repeat)

Process 2:

```
m = random();
```

```
array[index] = m;  
index++;
```

(repeat)

临界资源在哪里？



临界区在哪里？

剩余区在哪里？

接下来的问题：
如何解决数据访问冲突问题？

同濟大學軟件學院



【2016 统考真题】进程 P_1 和 P_2 均包含并发执行的线程，部分伪代码描述如下所示。

<pre>// 进程P1 int x=0; Thread1 () { int a; a=1; x += 1; } Thread2 () { int a; a=2; x += 2; }</pre>	<pre>// 进程P2 int x=0; Thread3 () { int a; a=x; x += 3; } Thread4 () { int b; b=x; x += 4; }</pre>
---	---

下列选项中，需要互斥执行的操作是 ()。

A. $a=1$ 与 $a=2$

B. $a=x$ 与 $b=x$

C. $x+=1$ 与 $x+=2$

D. $x+=1$ 与 $x+=3$

答案：C



同濟大學軟件學院

使用Win32 Threads

```
#include <windows.h>
#include <stdio.h>

// 全局变量
volatile int globalVariable = 0;
```

```
DWORD WINAPI incrementGlobalVariable(LPVOID threadId) {
    int i;
    for (i = 0; i < 1000000; i++)
        globalVariable++;
}

return 0;
}
```

```
int main() {
    HANDLE thread1, thread2;

    // 创建线程 1
    thread1 = CreateThread(NULL, 0, incrementGlobalVariable, NULL, 0, NULL);

    // 创建线程 2
    thread2 = CreateThread(NULL, 0, incrementGlobalVariable, NULL, 0, NULL);

    // 等待线程完成
    WaitForSingleObject(thread1, INFINITE);
    WaitForSingleObject(thread2, INFINITE);

    // 输出最终的全局变量值
    printf("Global variable: %d\n", globalVariable);

    // 清理线程资源
    CloseHandle(thread1);
    CloseHandle(thread2);

    return 0;
}
```

```
for (i = 0; i < 1000000; i++) {
    //globalVariable++;
    InterlockedIncrement(&globalVariable);
}
```

```
C:\Users\w7>C:\temp\wdq\debug\wdq.exe
Global variable: 2000000

C:\Users\w7>C:\temp\wdq\debug\wdq.exe
Global variable: 2000000

C:\Users\w7>C:\temp\wdq\debug\wdq.exe
Global variable: 2000000
```

还不如一个人干活快？

Linux下使用pthread线程库

```
#include <stdio.h>
#include <pthread.h>

// 全局变量
int globalVariable = 0;

// 线程函数
void incrementGlobalVariable(void *threadId) {
    for (int i = 0; i < 1000000; i++) {
        globalVariable++;
    }
}

int main() {
    // 创建两个线程
    pthread_t thread1, thread2;

    // 创建线程 1
    pthread_create(&thread1, NULL, incrementGlobalVariable, NULL);

    // 创建线程 2
    pthread_create(&thread2, NULL, incrementGlobalVariable, NULL);

    // 等待线程完成
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    // 输出最终的全局变量值
    printf("Global variable: %d\n", globalVariable);

    return 0;
}
```

```
C:\>C:\cygwin64\home\w7\c\a.exe
Global variable: 1105201
```

```
C:\>C:\cygwin64\home\w7\c\a.exe
Global variable: 1021934
```

```
C:\>C:\cygwin64\home\w7\c\a.exe
Global variable: 1156433
```


Python多线程

```
import threading

# 全局变量
global_variable = 0

# 线程函数
def increment_global_variable():
    for _ in range(1000000):
        global global_variable
        global_variable += 1

# 创建线程
thread1 = threading.Thread(target=increment_global_variable)
thread2 = threading.Thread(target=increment_global_variable)

# 启动线程
thread1.start()
thread2.start()

# 等待线程完成
thread1.join()
thread2.join()

# 输出最终的全局变量值
print("Global variable:", global_variable)
```



```
C:\temp>python thread.py
Global variable: 1254298

C:\temp>python thread.py
Global variable: 1243067

C:\temp>python thread.py
Global variable: 1287746

C:\temp>python thread.py
Global variable: 1193579

C:\temp>python thread.py
Global variable: 1255033
```



同濟大學軟件學院

```

public class MyThread {
    //声明共享变量
    private static int count = 0;

    public static void main(String[] args) {
        //线程函数
        Runnable incrementTask = () -> {
            for (int i = 0; i < 100000; i++) {
                count++;    // 自增1
            }
        };

        //创建两个线程
        Thread t1 = new Thread(incrementTask);
        Thread t2 = new Thread(incrementTask);
        //线程启动
        t1.start();
        t2.start();
        //等待线程结束
        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        //输出结果
        System.out.println("Count value: " + count);
    }
}

```

Java多线程



```

PS D:\wdqjava\thread>
56c486d53da45842\redhat
Count value: 146598
PS D:\wdqjava\thread>
56c486d53da45842\redhat
Count value: 159644
PS D:\wdqjava\thread>
56c486d53da45842\redhat
Count value: 149761
PS D:\wdqjava\thread>
56c486d53da45842\redhat
Count value: 157159
PS D:\wdqjava\thread>
paceStorage\1f73f7732bd
Count value: 165049

```



同濟大學軟件學院



数据竞争 (Data Race)



Data Race是指多个线程同时访问同一块数据，并且至少有一个线程是写操作，对数据的读取和修改产生了竞争，从而导致各种不可预计的问题。

拓展：了解ThreadSanitizer(tsan)线程数据竞争检查工具
其他Sanitizers (San)



常用方法



//方法1:使用互斥锁

```
pthread_mutex_lock(&lock);
```

```
for (int i = 0; i < 1000000; i++) {
```

//方法2:使用GCC的内置原子操作函数

```
__sync_fetch_and_add(&globalVariable,1);
```

```
globalVariable++;
```

```
}
```

```
pthread_mutex_unlock(&lock);
```

方法1: 互斥锁位置对性能有无影响? 如放在循环外还是内?

方法1和方法2哪个好? 为什么?

尝试用其他方法, 如自旋锁等。



同濟大學軟件學院



临界区问题/互斥问题



➤ 基本思路：
对临界区的访问**进行控制**

➤ 解决办法：

- 进入区
- 退出区
- 进去区和退出区位置

一个链表一个锁？一个节点一个锁？



同濟大學軟件學院



进程代码 — 临界区



do {

entry section

进入区

critical section

临界区

拓展：OSTEP第29章
Locked Data structures

exit section

退出区

remainder section

剩余区

} while (true);





进程代码 — 临界区



4、在下面对临界区的描述中，选择两条正确的论述

- (1) 指进程中用于实现进程互斥的那段代码
- (2) 进程中用于实现同步的那段代码
- (3) 进程中用于实现进程通信的那段代码
- (4) 进程中用于访问共享资源的那段代码
- (5) 进程中访问临界资源的那段代码
- (6) 若进程 A 与进程 B 必须互斥地进入自己的临界区，则进程 A 处于对应的临界区内时，仍能被 B 进程中断。
- (7) 若进程 A 与进程 B 必须互斥地进入自己的临界区，则进程 A 处于对应的临界区内时，不能被 B 进程中断。



进程代码 — 临界区



作业：

While (1) {

初始化： $a = b = x = y = 0$

启两个线程：

Thread1: $a = 1 \quad x = b$

Thread2: $b = 1 \quad y = a$

主线程循环执行以上两个子线程，并打印结果}

输出结果：

$\{x, y\} = \{(0, 1), (1, 0), (1, 1), (0, 0)\}$

解释不同输出结果的原因。

注： $(0, 0)$ 出现概率极低，需要增加循环次数



同濟大學軟件學院



进程代码 — 临界区



• a = 1 • x = b	• b = 1 • y = a	• a = 1 • x = b	• b = 1 • y = a	• a = 1 • x = b	• b = 1 • y = a	• x = b • y = a	• a = 1 • b = 1
x = 0	y = 1	x = 1	y = 1	x = 1	y = 1	x = 0	y = 0
• b = 1 • y = a	• a = 1 • x = b	• b = 1 • y = a	• a = 1 • x = b	• b = 1 • y = a	• x = b • y = a	• a = 1 • b = 1	• x = b • y = a
x = 1	y = 0	x = 1	y = 1	x = 1	y = 1	x = 0	y = 0



进程代码 — 临界区



初始状态: $A = \text{flag} = 0$;

T0

```
A = 1;  
flag = 1;
```

T1

```
While (flag == 0) ; //空循环  
Print(A)
```

知乎 @shawn

会不会有打印出 $A=0$ 的结果？

内存模型



同濟大學軟件學院



临界区问题/互斥问题



➤ 使用原则

- 互斥访问（临界资源访问特性）
- 有空让进（资源利用率）
- 无空等待
- 多中择一
- 有限等待（性能）
- 让权等待（硬件做不到，需要OS干预）

处于等待状态的进程应放弃占用CPU，以使其他进程有机会得到CPU的使用权。对于单处理器非抢占式是系统必须满足，但是多核多处理器就不是必须遵守的原则了。





临界区问题/互斥问题



- 软件方法
 - 基本思路
 - 算法1
 - 算法2
 - 算法3
 - 算法4





临界区问题/互斥问题



➤ 软件方法

- 基本思路
- 算法1
- 算法2
- 算法3
- 算法4





临界区问题/互斥问题



■ 软件方法基本思路

- 在进入区检查和设置一些**标志**，如果已有进程在临界区，则在进入区通过循环检查进行等待；在退出区修改标志
- 主要问题是**设置什么标志**和**如何检查标志**
- 为了简单说明，算法中只涉及**两个**协作进程





临界区问题/互斥问题



➤ 软件方法

- 基本思路
- 算法1
- 算法2
- 算法3
- 算法4



同濟大學軟件學院



临界区问题/互斥问题



■ 算法1：单标志位

设有两个进程 P_i , P_j , 其中 P_i 进程设立一个**公用整型变量** $turn$, 用来描述允许进入临界区的进程标识。

- 在**进入区**循环检查是否允许本进程进入
- $turn$ 为 i 时, 进程 P_i 可进入
- 在**退出区**修改允许进入进程标识
- 进程 P_i 退出时, 改 $turn$ 为进程 P_j 的标识 j ;





临界区问题/互斥问题



■ 算法1：单标志位

```
while (turn != i);  
    critical section  
turn = j;  
    remainder section
```

忙等

+



临界区问题/互斥问题



```
void* incrementGlobalVariable1(void *threadId) {
    for (int i = 0; i < 1000000; i++) {
        while(turn!=1);
        globalVariable++;
        turn = 2;
    }
    return NULL;
}

void* incrementGlobalVariable2(void *threadId) {
    for (int i = 0; i < 1000000; i++) {

        while(turn!=2);
        globalVariable++;
        turn = 1;
    }
    return NULL;
}
```

问题解决了！ ?

```
C:\Users\w7>C:\cygwin64\ho
Global variable: 2000000
C:\Users\w7>C:\cygwin64\ho
Global variable: 2000000
C:\Users\w7>C:\cygwin64\ho
Global variable: 2000000
C:\Users\w7>C:\cygwin64\ho
Global variable: 2000000
C:\Users\w7>C:\cygwin64\ho
Global variable: 2000000
```



同濟大學軟件學院



临界区问题/互斥问题



有问题吗？

✓ 缺点：进程**强制轮流**进入临界区，容易造成资源利用不充分。

如：在 P_i 让出临界区之后， P_j 使用临界区之前， P_i 不可能再次使用临界区。随着进程数增多，问题会更明显，**资源利用率会降低**。



同濟大學軟件學院



临界区问题/互斥问题



➤ 软件方法

- 基本思路
- 算法1
- 算法2
- 算法3
- 算法4





临界区问题/互斥问题



■ 算法2：双标志位（先检查）

设立一个标志数组flag[]：描述进程是否在临界区，初值均为FALSE（未使用）。如

flag[i]=FALSE //Pi未进入临界区。

- 先检查，后修改

在**进入区**检查另一个进程是否在临界区，不在时修改本进程在临界区的标志；

- 在**退出区**修改本进程在临界区的标志；



临界区问题/互斥问题



```
while (flag[j]);           <a>  
flag[i] = TRUE;           <b>
```

critical section

```
flag[i] = FALSE;
```

remainder section

✓ 优点

不用交替进入，可连续使用

有问题吗？

同濟大學軟件學院





临界区问题/互斥问题



✓ 缺点

P_i 和 P_j 可能同时进入临界区

如：按下面序列执行时，会同时进入：

“ $P_i \langle a \rangle P_j \langle a \rangle P_i \langle b \rangle P_j \langle b \rangle$ ”。即在检查对方flag之后和切换自己flag之前有一段时间，结果都检查通过。





临界区问题/互斥问题



- 软件方法
 - 基本思路
 - 算法1
 - 算法2
 - 算法3
 - 算法4





临界区问题/互斥问题



■ 算法3：双标志位（先修改）

类似于算法2，区别在于先修改后检查

```
flag[i] = TRUE;                <b>
```

```
while (flag[j]);               <a>
```

```
critical section
```

```
flag[i] = FALSE;
```

```
remainder section
```





临界区问题/互斥问题



✓ 优点

防止两个进程同时进入临界区

✓ 缺点：

P_i 和 P_j 可能都进入不了临界区

如：按下面序列执行时，都进不了临界区：

“ $P_i < b > P_j < b > P_i < a > P_j < a >$ ”。即在切换自己flag之后和检查对方flag之前有一段时间，结果都切换flag，都检查不通过。

算法2,3问题出在：检查和修改操作不能连续进行。



同濟大學軟件學院



临界区问题/互斥问题



➤ 软件方法

- 基本思路
- 算法1
- 算法2
- 算法3
- 算法4 (Peterson算法)





临界区问题/互斥问题



- 算法4：结合算法1 (turn) 和算法3 (flag[])
 - 正确的算法
 - 先修改、后检查、后修改者等待
 - $turn=j$; 描述可进入的进程 (同时修改标志位)
 - 在进入区先修改后检查，并检查并发修改的先后
 - 检查对方flag，如果不在临界区则自己进入
 - 空闲则入
 - 再检查turn：保存的是较晚的一次赋值，则较晚的进程等待，较早的进程进入
 - 先到先入，后到等待





临界区问题/互斥问题



算法4：结合算法1和算法3

```
flag[i] = TRUE; turn = j; 顺序!  
while (flag[j] && turn == j);
```

critical section

```
flag[i] = FALSE;
```

remainder section

Pi	Pj
flag[i]=true;	flag[j]=true;
Turn=j;	Turn=i;

发生竞争! 假设Pi后修改
turn=j

阻塞

进入临界区

flag[j]=false;

有可能进入



同濟大學軟件學院



临界区问题/互斥问题



算法4: Peterson算法

- 只有一个进程修改flag为true, 则它可进;
- 两个进程都改flag为true, 先改turn的能进, 另一个等待

思考: flag和turn的修改顺序能不能调换? 课后用程序验证该算法是否有效。

如果失效, 分析失效原因。

无法运行在现代硬件(松散内存一致性模型)上。



同濟大學軟件學院



临界区问题/互斥问题



算法4：加了内存屏障的Pettersen算法

```
flag[i] = TRUE; turn = j; __sync_synchronize();  
while (flag[j] && turn == j);
```

critical section

```
flag[i] = FALSE;
```

remainder section





临界区问题/互斥问题



问题：Pettersson算法中，turn变量的作用是：

- A、表示轮到哪个线程进入临界区
- B、表示哪个线程先发出访问请求
- C、表示哪个线程后发出访问请求
- D、表示哪个线程已进入临界区

答案：A





临界区问题/互斥问题



■ 软件算法存在如下缺点：

- 忙等待（做不到让权等待）
- 实现过于复杂，需要高的编程技巧

如“Dekker算法”（解决了忙等问题）、
“面包店算法”（适用于多进程）等其他
纯软件算法。



忙等



- 单核单处理器的忙等是**致命的**
- 多处理下忙等是可以接受的，甚至有时候可以提高效率，例如自旋锁





临界区问题/互斥问题



➤ 硬件方法（原子操作）

- “开关中断” 指令
- Test_and_set “测试并设置” 指令 (TAS)
- Compare_and_swap “交换” 指令 (CAS)
- MIPS的LL/SC指令对 (OSTEP28.11)

拓展阅读OSTEP-ch28lock



同濟大學軟件學院



临界区问题/互斥问题



■ “开关中断”指令

- 进入临界区前执行：执行“关中断”指令
- 离开临界区后执行：执行“开中断”指令
- 一般只留给OS自己使用（可能丢掉中断）
- 适用于单处理器
- 每个处理器一个中断

不适用通用同步方案

```
1 void lock() {  
2     DisableInterrupts();  
3 }  
4 void unlock() {  
5     EnableInterrupts();  
6 }
```



同濟大學軟件學院



临界区问题/互斥问题



■ Test-and-Set指令

```
boolean test_and_set (boolean *target)
{
    boolean rv = *target;
    *target = TRUE;
    return rv;
} //atomically原子地
```

说明: *target: FALSE 能进 (初始化)

TRUE 不能进

对返回结果rv进行测试, 对*target进行设置



同濟大學軟件學院



临界区问题/互斥问题



- Test-and-Set指令实现互斥

```
Boolean lock = false; //互斥锁
```

```
do {
```

```
    while (test_and_set(&lock))
```

```
        ; /* do nothing自旋等待*/
```

```
    /* critical section */
```

```
    lock = false;
```

```
    /* remainder section */
```

```
} while (true);
```





临界区问题/互斥问题



■ Compare_and_swap指令

```
int compare_and_swap(int *value, int expected, int
new_value) {
    int temp = *value;
    if (*value == expected)//判断有没有其他任务改过
        *value = new_value; //没改过才更新
    return temp;
} //atomically原子地
```

拓展阅读Disruptor开源项目中是如何利用CAS指令实现“无锁”从而提高性能的。

同濟大學軟件學院





临界区问题/互斥问题



- Compare_and_swap指令实现互斥

```
int lock = 0;

do {
    while (compare_and_swap(&lock, 0, 1) != 0)
        ; /* do nothing 自旋等待*/
    /* critical section */
    lock = 0;
    /* remainder section */
} while (true);
```





临界区问题/互斥问题



■ 硬件方法优点如下：

- 适用于任意数目的进程，在单处理器或多处理器上均适用
- 简单，容易验证其正确性
- 可以支持进程内存的多个临界区，只需为每个临界区设立一个布尔变量





临界区问题/互斥问题



- 硬件方法缺点如下：
 - 等待要耗费CPU时间，不能实现“让权等待”
 - 可能“饥饿”：从等待进程中随机选择一个进入临界区，有的进程可能一直选不上
 - 可能死锁
 - 用户程序一般不能直接使用
- 互斥锁（OS对硬件指令的封装，解决忙等问题）



使用Win32 Threads

```
#include <windows.h>
#include <stdio.h>

// 全局变量
volatile int globalVariable = 0;
```

```
DWORD WINAPI incrementGlobalVariable(LPVOID threadId) {
```

```
    int i ;
    for (i = 0; i < 1000000; i++)
        globalVariable++;
}
```

```
    return 0;
```

```
}
```

```
int main() {
    HANDLE thread1, thread2;
```

```
    // 创建线程 1
```

```
    thread1 = CreateThread(NULL,
```

```
    // 创建线程 2
```

```
    thread2 = CreateThread(NULL,
```

```
    // 等待线程完成
```

```
    WaitForSingleObject(thread1,
```

```
    WaitForSingleObject(thread2,
```

```
    // 输出最终的全局变量值
```

```
    printf("Global variable: %d\n", globalVariable);
```

```
    // 清理线程资源
```

```
    CloseHandle(thread1);
```

```
    CloseHandle(thread2);
```

```
    return 0;
```

```
}
```

```
    for (i = 0; i < 1000000; i++) {
        //globalVariable++;
        InterlockedIncrement(&globalVariable);
    }
```

```
C:\Users\w7>C:\temp\wdq\debug\wdq.exe
Global variable: 960880
```

```
C:\Users\w7>C:\temp\wdq\debug\wdq.exe
Global variable: 2000000
```

```
C:\Users\w7>C:\temp\wdq\debug\wdq.exe
Global variable: 2000000
```

```
C:\Users\w7>C:\temp\wdq\debug\wdq.exe
Global variable: 2000000
```



Ch 7: Process Synchronization



- Background
- 临界区问题/互斥问题
- **进程同步**
- 典型同步互斥问题
- 举例



进程同步



✚ 同步机制

✚ 信号量

✚ 管程



进程同步



✚ 同步机制

✚ 信号量

✚ 管程



进程同步



➤ 同步机制

- OS可从进程管理者的角度来处理互斥同步问题，这就是**同步机制**。
- 同步机制：
 - 自旋锁
 - 互斥锁
 - 条件变量
 - 信号量P、V操作**
 - 管程（资源管理模块）
 -



同濟大學軟件學院

进程同步



✚ 同步机制

✚ 信号量

✚ 管程

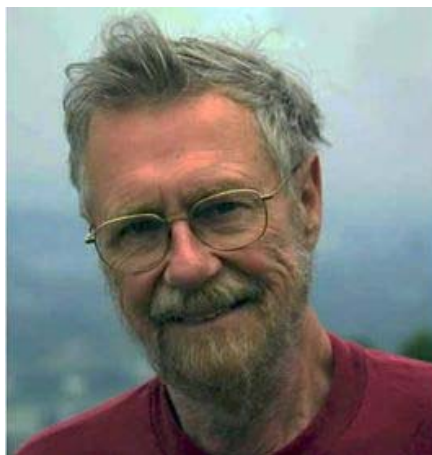


信号量的提出



➤ 信号量

是一种卓有成效的进程同步机制。1965年，荷兰学者Dijkstra提出的。



同濟大學軟件學院



信号量



- ✓ 整形信号量（不构成阻塞队列，忙等）
例如：自旋锁
- ✓ 记录型信号量
- ✓ “信号量集” 机制（解决死锁问题）
如：P (s1, s2, ...)



记录型信号量



➤ 信号量 (semaphore)

- ✓ 信号量表示资源的实体，一个与**队列**有关的**整型**变量。
- ✓ 其数据结构定义如下：

```
typedef struct{  
    int value;  
    struct process *list; //阻塞在该信号量的进程队列  
} semaphore;
```



信号量



✓ 信号量的物理含义：

$S > 0$ 表示有 S 个资源可用

$S = 0$ 表示无资源可用

$S < 0$ 则 $|S|$ 表示 S 等待队列中的进程个数



信号量



✓ 信号量说明：

semaphore s;

信号量只能通过**初始化**和**P/V原语**来访问，不受进程调度的打断。





P/V操作



✓ P原语 (wait, down)

```
wait(semaphore *S) {  
    S->value--;  
    if (S->value < 0) {  
        add this process to S->list;  
        block();  
    }  
}
```

✓ P操作用于申请资源（或使用权），进程执行P原语时，可能阻塞自己；

同濟大學軟件學院



P/V操作



✓ P原语 (wait , down)

执行P操作意味着申请分配一个单位的资源

P (S)

① $S=S-1$

成功 → ② 如果 $S \geq 0$, 则调用P(S)的进程继续运行

不成功 → ③ 如果 $S < 0$, 则调用P(S)的进程被阻塞, 并把它插入到等待信号量S的阻塞队列中。



同濟大學軟件學院

P/V操作



✓ V原语 (signal , up)

```
signal(semaphore *S) {  
    S->value++;  
    if (S->value <= 0) {  
        remove a process P from S->list;  
        wakeup(P);  
    }  
}
```

✓ V操作用于释放资源（或使用权），
执行V原语时**可能需要唤醒**一个阻塞进程。

同濟大學軟件學院





P/V操作



✓ V原语 (signal , up)

执行V操作意味着释放一个单位的资源

V(S)

- ① $S = S + 1$
- ② 如果 $S > 0$, 则调用V(S)的进程继续运行
- ③ 如果 $S \leq 0$, 则从等待信号量s的阻塞队列中唤醒头一个进程到就绪队列中, 然后调用V(S)的进程继续运行。



同濟大學軟件學院

表 31.2

追踪线程：两个线程使用一个信号量 二值信号量：初值为1（互斥锁）

值	线程 0	状态	线程 1	状态
1		运行		就绪
1	调用 sem_wait()	运行		就绪
0	sem_wait()返回	运行		就绪
0	（临界区：开始）	运行		就绪
0	中断；切换到→T1	就绪		运行
0		就绪	调用 sem_wait()	运行
-1		就绪	sem 减 1	运行
-1		就绪	(sem<0) →睡眠	睡眠
-1		运行	切换到→T0	睡眠
-1	（临界区：结束）	运行		睡眠
-1	调用 sem_post()	运行		睡眠
0	增加 sem	运行		睡眠
0	唤醒 T1	运行		就绪
0	sem_post()返回	运行		就绪
0	中断；切换到→T1	就绪		运行
0		就绪	sem_wait()返回	运行
0		就绪	（临界区）	运行
0		就绪	调用 sem_post()	运行
1		就绪	sem_post()返回	运行



信号量的P/V操作



✓ P、V操作必须成对出现

- 当为**互斥操作**时，它们同处于**同一**进程
- 当为**同步操作**时，则在**不同**进程中出现

如果P(S1)和P(S2)两个操作在一起，
那么P操作的**顺序**至关重要。



信号量



✓ P/V优缺点

- 标记型信号量可以避免忙等
- 简单，而且表达能力强
- 不够安全，使用不当会出现死锁；
- 遇到复杂同步互斥问题时不方便



信号量



✓ OS利用**信号量**对进程和资源进行控制和管理

✓ 信号量分类

- **公用信号量**用来实现进程间的互斥，初值为1，允许它所联系的一组进程对它执行P/V操作
- **私用信号量**用来实现进程间的同步，初值为0或者某个正整数，仅允许拥有它的进程对其执行P/V操作。

✓ 信号量的值

- 信号量取值为非负值表示当前空闲资源数，若为负值其绝对值表示当前等待临界区的进程数。



同濟大學軟件學院

信号量类型



- ✓ 互斥信号量用于申请或释放资源的使用权，常初始化为1。 资源信号量用于申请或归还资源，可以

初始化为大于1的正整数，表示系统中某类资源的可用数。

- ✓ 互斥锁

互斥锁是一种互斥信号量，可能阻塞任务
自旋锁不阻塞，会忙等





P/V操作的应用



➤ P/V操作应用1：实现互斥

- 为临界资源设置一个互斥信号量mutex，初值为1；
- 在每个进程中，将临界区代码置于P(mutex)和V(mutex)原语之间
- 必须成对使用P和V原语：

遗漏P原语则不能保证互斥访问，遗漏V原语则不能在使用临界资源之后将其释放（给其他等待的进程）





P/V操作的互斥应用



P(mutex) ;

critical section

V(mutex) ;

remainder section





P/V操作的应用



➤ P/V操作应用2：实现同步

■ 前趋关系

并发执行的进程P1和P2中，分别有代码C1和C2，要求C1在C2开始前完成，那么C1就是C2的前趋；

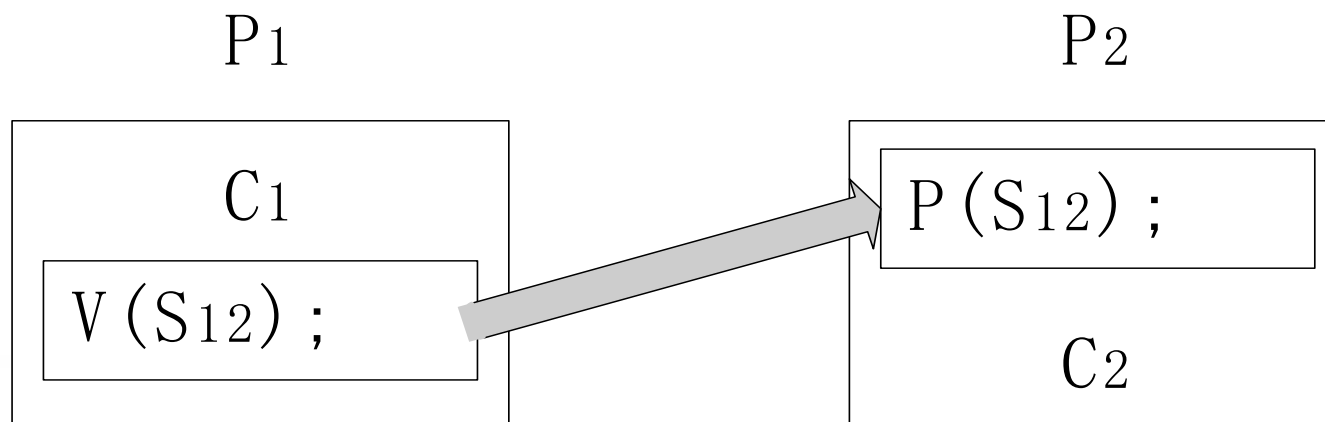
如何利用信号量保证前趋执行呢？



P/V操作的同步应用



如何初始化信号量？



设置一个同步信号量 S_{12} ，其初值为0。



同濟大學軟件學院



练习



● 设有 n 个进程共享一个互斥段，对于如下两种情况：

- (1) 如果每次只允许一个进程进入互斥段。
- (2) 如果最多允许 m 个进程 ($m < n$) 同时进入互斥段。

试问：所采用的互斥信号量初值是否相同？

信号量值的变化范围如何？

答：初值是不相同的，(1) 其中的初值是1，变化范围是 $[1-n, 1]$ ，(2) 中的初值是 m ，变化范围是 $[m-n, m]$ 。



进程同步



✚ 同步机制

✚ 信号量

✚ 管程





管程



- + 管程的引入
- + 管程概念
- + 管程的构成
- + 管程的基本特性
- + 条件变量





管程



- ✚ 管程的引入
- ✚ 管程概念
- ✚ 管程的构成
- ✚ 管程的基本特性
- ✚ 条件变量





管程的引入



■ 问题：

信号量机制的引入解决了进程同步的描述问题，但信号量的大量同步操作**分散**在各个进程中不便于管理，还可能导致系统**死锁**。



管程的引入



■ 引入：

Dijkstra于1971年提出：把所有进程对某一种临界资源的同步操作都**集中**起来，构成一个所谓的**秘书进程**。凡要访问该临界资源的进程，都需先报告秘书，由秘书来实现诸进程对同一临界资源的互斥使用。

1973年Hansen等人将秘书进程的思想发展为**管程**概念。





管程 (monitor)



- + 管程的引入
- + **管程概念**
- + 管程的构成
- + 管程的基本特性
- + 条件变量



管程的概念



■ 定义：

Hansen为管程所下的定义：管程定义了一个**数据结构**和能为并发进程所执行的**一组操作**，这组操作能同步进程和改变管程中的数据。





管程



- ✚ 管程的引入
- ✚ 管程概念
- ✚ 管程的构成
- ✚ 管程的基本特性
- ✚ 条件变量

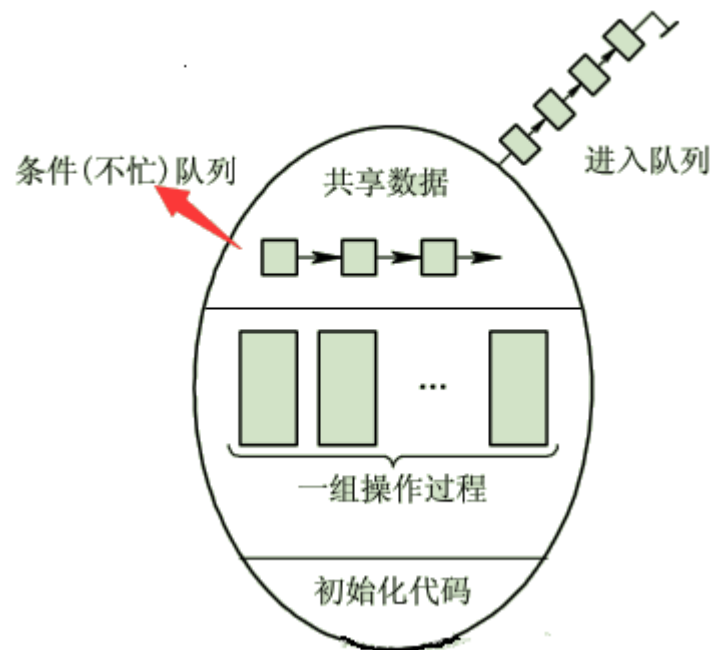


管程的构成



三部分内容：

- 局部于管程的
共享数据结构
- 对共享数据结构
进行操作的
一组函数
- 对局部于管程
的数据**设置初
始值**的语句



管程相当于围墙，把共享数据和操作他们的函数围起来，每次**只允许一个**进程进入，从而实现互斥（管程是**编程语言层面**的，如C、C++不支持，JAVA、C#支持）。

同濟大學軟件學院





管程



- ✚ 管程的引入
- ✚ 管程概念
- ✚ 管程的构成
- ✚ **管程的基本特性**
- ✚ 条件变量





管程的基本特征



- **局部性**：局部于管程的数据只能被局部于管程内的函数所访问。
- **保护性**：一个进程只有通过调用管程内的函数才能进入管程访问共享数据。
- **互斥性**：每次仅允许一个进程在管程内执行某个函数。
- **系统级**：由于管程是一个**语言成分**，所以管程的互斥访问完全由编译程序在编译时自动添加上，无需程序员关心，而且保证正确。

作业：可尝试使用JAVA的管程机制完成生产者-消费者问题。



同濟大學軟件學院



管程



- + 管程的引入
- + 管程概念
- + 管程的构成
- + 管程的基本特性
- + 条件变量



条件变量



- 对于不直接支持管程的语言，可使用**互斥量**和**条件变量**及操作原语来实现管程（如C++）
 - ✓ **条件变量**用于区别各种不同的等待原因。
 - ✓ **同步原语**(wait和signal)。





Ch 7: Process Synchronization



- Background
- 临界区问题/进程互斥
- 进程同步
- 典型同步互斥问题
- 举例





典型同步互斥问题



- 生产者－消费者问题
- 读者－写者问题
- 哲学家就餐问题





典型同步互斥问题



- 生产者－消费者问题
- 读者－写者问题
- 哲学家就餐问题





生产者-消费者问题



背景

- ✓ 也叫有界缓冲区 (bounded buffer) 问题
- ✓ 由Dijkstra最早提出
- ✓ 研究该问题时提出了信号量、管程等机制



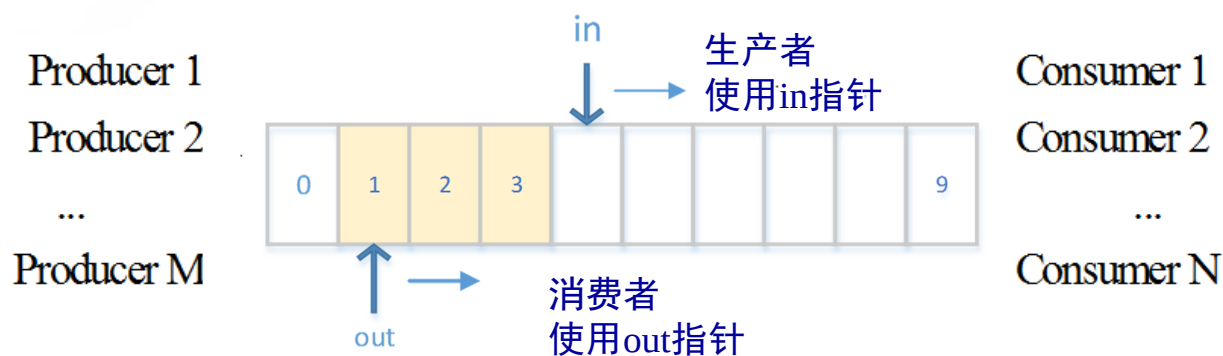
同濟大學軟件學院

生产者-消费者问题



问题描述

- ✓ 若干进程通过**有限的共享缓冲区**交换数据；
- ✓ “**生产者**”进程不断放入数据，
而“**消费者**”进程不断获取数据；
- ✓ 共享缓冲区共有 N 个；**任何时刻只能有一个**进程可对共享缓冲区进行操作。



同濟大學軟件學院





生产者-消费者问题



- ✓ 生产指针in和消费指针out初始化指向缓冲区的第一个存储单元（如何进一步优化？）
- ✓ 生产者向存储单元一次存放一条数据，且in指针向后移一个位置。
- ✓ 消费者从缓冲区逐条取数据，相应存储单元变为“空”，out指针向后移动一个单元位置。
- ✓ in、out位置取模----环形buffer
- ✓ 不控制，两者会发生什么问题？
- ✓ 现实中遇到的大多数同步互斥问题都是生产者-消费者问题的变型





生产者 - 消费者问题



✓ 初步方案

- full是“满”数目，初值为0，empty是“空”数目，初值为N。
- $full + empty == N$



生产者-消费者问题



Producer

```
P(empty);  
    one unit → buffer;  
V(full);
```

Consumer

```
P(full);  
    one unit ← buffer;  
V(empty);
```

empty、full：同步信号量（在这里也可以叫资源信号量）

生产和消费逻辑参见之前Ch4中的介绍，只是不用再做判空和判满操作了。

有问题吗？

思考：如果buffer大小为1还有问题吗？

答案：不会有问题。（此时信号量也起到互斥作用，优化后可以只使用一个信号量）

同濟大學軟件學院





生产者 - 消费者问题



➤ 混乱现象

- ✓ 生产者和消费者可能同时进入缓冲区，甚至同时读/写一个存储单元，将导致结果不确定（数据覆盖）
- ✓ 所以必须使生产者和消费者**互斥**进缓冲区，即某一时刻只允许一个实体（生产者或消费者）访问缓冲区。
- ✓ 生产者不能向满缓冲区放数据，消费者也不能在空缓冲区中取数据，两者必须**同步**





生产者 - 消费者问题



✓ 问题解决

- full是“满”数目，初值为0，empty是“空”数目，初值为N。
- $full + empty == N$
- mutex用于访问缓冲区时的互斥，初值是1



生产者 - 消费者问题



Producer

```
P(mutex);  
P(empty);  
    one unit --> buffer;  
V(full);  
V(mutex);
```

Consumer

```
P(mutex);  
P(full);  
    one unit <-- buffer;  
V(empty);  
V(mutex);
```

mutex: 互斥信号量

empty、full: 初值分别为n、0, 同步信号量 (资源信号量)

场景: 满时生产者获得mutex; 空时消费者获得mutex

结果: 死锁

P操作的顺序!!!

同濟大學軟件學院





生产者 - 消费者问题



Producer

同步在外，互斥在内。
减小互斥范围

Consumer

```
P(empty);  
P(mutex);  
    one unit → buffer;  
V(mutex);  
V(full);
```

```
P(full);  
P(mutex);  
    one unit ← buffer;  
V(mutex);  
V(empty);
```

mutex: 互斥信号量

拓展: 高性能无锁并发框架--Disruptor

empty、full: 初值分别为n、0, 同步信号量 (资源信号量)

- 思考:
- 1 有多个生产者和消费者, 缓冲区容量为1时; (分水果)
 - 2 只有一个生产者和多个消费者, 缓冲区容量为1时; (吸烟者)
 - 3 只有一个生产者和消费者, 缓冲区容量为N时;

.....

同濟大學軟件學院





生产者-消费者问题



✓ linux下的“条件变量”机制等

课后尝试利用“条件变量”完成生产者-消费者的合作问题。

信号量: `sem_XXX`

条件变量: `pthread_cond_XXX`

互斥锁: `pthread_mutex_XXX`

自旋锁: `spin_lock/unlock`





典型同步互斥问题



- ✚ 生产者 - 消费者问题
- ✚ 读者 - 写者问题
- ✚ 哲学家就餐问题





读者-写者问题



➤ 读者写者问题 (1971年Courtois提出)

✓ 问题描述

有两组并发进程：**读者和写者**，共享一组数据区。

✓ 要求：

允许多个读者同时可执行读操作（提高读效率）

不允许读者、写者同时操作

不允许多个写者同时操作



同濟大學軟件學院



读者-写者问题



➤ 读者写者问题

- ✓ 采用生产者/消费者问题解决，严格互斥任何读者和写者进程，能保证数据的更新操作是正确的
- ✓ 但对于读者进程，如果被严格互斥，则严重影响系统效率（例如数据库查找等）
- ✓ 如果一个写者进程正在修改数据，别的写者以及任何读者都不能访问数据





读者－写者问题



- ✓ 问题分析（多种变型）
 - 第一类：读者优先（重点）
 - 第二类：写着优先
 - 第三类：公平竞争（自行讨论）



读者-写者问题



- 第一类：**读者优先**(最简单)
 - 无读者、写者可写
 - 有写者写，读者等
 - **有读者在读，写者等待，但此时新读者可以进行读，写者接着等待（读者可以插队）**
 - **最后一个读者完成访问后，发现写者在等，则唤醒其中一个写着。**
 - 写者完成访问后，发现有等待者，按照FIFO或者其它原则唤醒写者或者所有读者（注意此时并没强调读者优先）
 - 读者优先可能导致**写者饥饿**（怎么解决？）



第一类 读者优先



读者：

```
while (true) {  
    P(rcountmutex);  
    readcount ++;  
    if (readcount==1)  
        P (wmutex);  
    V(rcountmutex);  
    读  
    P(rcountmutex);  
    readcount --;  
    if (readcount==0)  
        V(wmutex);  
    V(rcountmutex);  
};
```

写者：

```
while (true) {  
  
    P(wmutex);  
    写  
    V(wmutex);  
  
};
```

同濟大學軟件學院





第一类 读者优先



✓ 说明：

- readcount用来记录当前有多少个读者在访问数据，初值为0（它不是信号量）
- rcountmutex、wmutex初值为1
- rcountmutex用来保证读者之间互斥地修改readcount。
- wmutex是读者和写者公用的互斥信号量，用来互斥读写同时进行（第一个读者和写者竞争）





第二类 写者优先



- 第二类：写者优先
 - 多个读者可以同时进行读
 - 写者必须互斥，即只允许一个写者写
 - 写者优先的附加限制：如果一个读者申请进行读操作时已有另一写者在等待访问共享资源，则该读者必须等到没有写者处于等待状态后才能开始读操作。（不允许读者插队，唤醒时优先唤醒写者）



第二类 写者优先



读者：

```
while (true) {  
    P(z);  
    P(rMutex);  
    P(rCountMutex);  
    readcount++;  
    if(readcount==1) P(w);  
    V(rCountMutex);  
    V(rMutex);  
    V(z);  
    读  
    P(rCountMutex);  
    readcount--;  
    if(readcount==0) V(w);  
    V(rCountMutex);  
};
```

写者：

```
while (true) {  
    P(wCountMutex);  
    writecount++;  
    if(writecount==1) P(rMutex);  
    V(wCountMutex);  
    P(w);  
    写  
    V(w);  
    P(wCountMutex);  
    writecount--;  
    if(writecount==0) V(rMutex);  
    V(wCountMutex);  
};
```

问题：z信号量的作用？ 116

同濟大學軟件學院



第二类 写者优先



说明

- readcount, writecount 用来记录当前有多少个读者和写者在访问数据, 初值为0
- rCountMutex, wCountMutex, rMutex, w 初值为1
- z 初值为n
- w 是读者和写者公用的互斥变量, 用来互斥读写或者写写同时进行
- rCountMutex, wCountMutex 是保证多个读者或多个写者对 readcount 和 writecount 互斥访问
- 信号量 rMutex 用来保证有写进程等待时, readcount 不增加
- 信号量 z (不是必须的) 主要作用是减少对 rMutex 的争用, 提高读者之间的并发性。





第二类 写者优先



✓ 信号量rMutex 的必要性

rMutex在写者代码中：

- 第一个写者会P(rMutex)。
- 最后一个写者会V(rMutex)。

这意味着当有写者等待或正在写时，rMutex被持有，阻止新的读者进入。（写者优先）

rMutex在读者代码中：

读者需要先获取rMutex才能继续。rMutex的作用是：

- 防止在有写者等待时，新的读者开始读。（写者优先）。

第三类 读者写者公平竞争（自行讨论）

同濟大學軟件學院



第二类 写者优先



✓ linux下的“读写锁”机制

读写锁状态	以共享方式获取 (读者)	以独占方式获取 (写者)
自由	成功	成功
共享	成功	等待
独占	等待	等待

课后尝试，利用读写锁分别完成写者优先、读者优先和平等分配问题。



同濟大學軟件學院



典型同步互斥问题



- 生产者－消费者问题
- 读者－写者问题
- 哲学家就餐问题





哲学家就餐问题

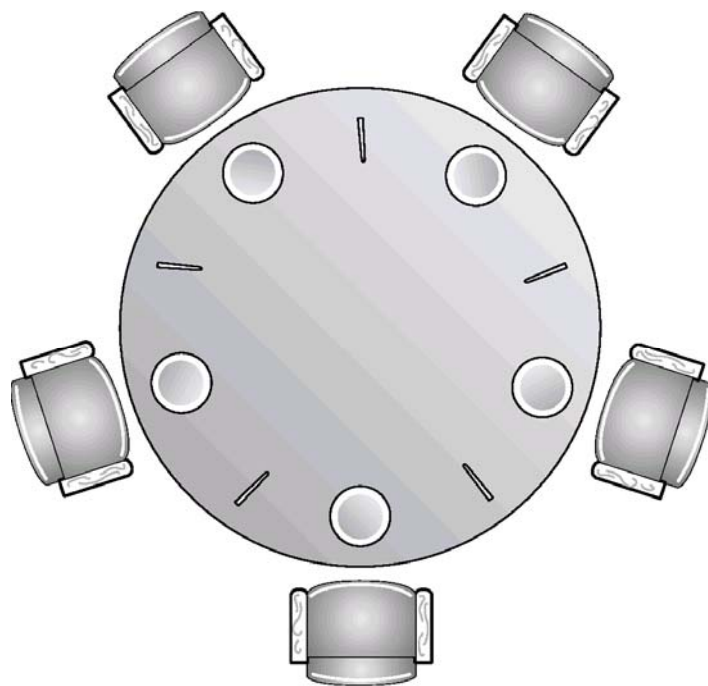


➤ 问题描述

- ✓ 有五个哲学家围坐在一圆桌旁，桌中央有一盘通心粉，每人面前有一只空盘子，每两人之间放一只筷子。
- ✓ 每个哲学家的行为是思考，感到饥饿，然后吃通心粉。
- ✓ 为了吃通心粉，每个哲学家必须拿到两只筷子，并且每个人只能直接从自己的左边或右边去取筷子。



哲学家就餐问题



同濟大學軟件學院

哲学家就餐问题



✓ 问题解决

- 为每根筷单独设立一个信号量，哲学家取筷子执行P操作，放下筷子执行V操作。

var chopstick: array[0..4] of semaphore; 初值为1

- 各个哲学家执行的程序为：

P(chopstick[i]); //取左边筷子

P(chopstick[(i+1) mod 5]); //取右边筷子

进食

V(chopstick[(i+1) mod 5]); //放下右边筷子

V(chopstick[i]); //放下左边筷子

思考

存在什么问题？



同濟大學軟件學院



哲学家就餐问题



✓ 算法改进1

- 当哲学家获得左边筷子后，查看右边筷子是否可用，如不可用，则“谦让”，放下已获得的左边筷子，等一段时间再重复之前的过程。（可能出现“活锁”）
- 该方法可能导致各哲学家都“谦让”，都处于“忙等”，并都进入“饥饿”状态。





哲学家就餐问题



✓ 算法改进2

- 再买1支筷子—成本增加

✓ 算法改进3

- 增加一个服务员，让他安排其中的四位哲学家先入座，四位竞争5支筷子，必有一位会同时获得两双筷子，他用餐完毕，再安排剩下那位就坐，重复竞争、用餐过程。





哲学家就餐问题



var chopstick: array[0..4] of semaphore;初值为1

room: semaphore ;初值为4

各个哲学家执行的程序为:

P(room);

P(chopstick[i]); //取左边筷子

P(chopstick[(i+1) mod 5]); //取右边筷子

进食

V(chopstick[(i+1) mod 5]); //放下右边筷子

V(chopstick[i]); //放下左边筷子

V(room)

思考

同濟大學軟件學院





哲学家就餐问题



✓ 算法改进4

- 使用非对称解决：

- ✓ 奇数哲学家先拿起左边筷子，再拿右边筷子；
- ✓ 偶数哲学家先拿起右边筷子，再拿左边筷子；

✓ 算法改进5

- 使用信号量集：同时申请左右两个筷子。

✓ 算法改进6

- 对资源（筷子）进行编号：申请必须按序申请。

.....

同濟大學軟件學院





信号量使用注意事项



➤ 总结

- 进程应该先申请资源信号量，再申请互斥信号量，顺序不能颠倒--同步在外，互斥在内；
- 对任何信号量的P/V操作必须**成对**使用。同一进程中的多对P/V语句只能嵌套，**不能交叉**；
- 对同一信号量的P/V操作可以不在同一进程中；
- P/V语句不能颠倒**顺序**。





Ch 7: Process Synchronization



- Background
- 临界区问题/进程互斥
- 进程同步
- 典型同步互斥问题
- 举例



练习



问题描述：某小超市可容纳50人同时购物。入口处有篮子，每个购物者从入口处拿一只篮子进入购物。出口处结账（入口和出口是两个门），并归还篮子（出、入口禁止多人同时通过），用信号量P，V写出购物者的同步算法。

需要几个信号量？几个进程？试写出进程流程。



练习



● 参考答案

信号量：(超市剩余空间) $empty=50$,
(入口) $mutex1=1$, (出口) $mutex2=1$

```
1  shopper(){
2      while(1){
3          P(empty);          /*这里的P操作可以理解成请求, 申请一个空闲区域*/
4          P(mutex1);
5          从入口进入, 取一只篮子;      /*入口只能一个人通过, 该动作执行的过程中, 不允许其他进程进入*/
6          V(mutex1);
7          购物;
8          P(mutex2);
9          从入口出去, 放下篮子;
10         V(mutex2);
11         V(empty);          /*这里的V操作可以理解成归还, 释放一个空闲区域*/
12     }
13 }
```



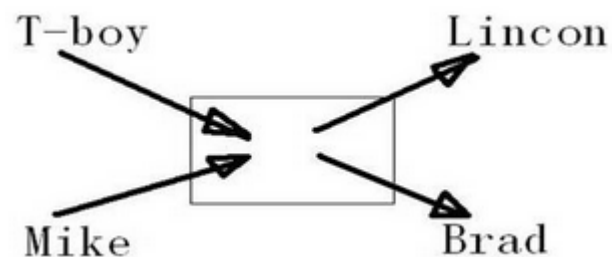
练习



● 寄信

问题描述：

T-boy给brad送信，Mike给Lincon送信，但他们送信收信都通过同一个树洞，一次只能放一封信。



需要几个信号量？几个进程？试写出各进程流程。



同濟大學軟件學院

练习



● 寄信-参考答案

信号量： Lincon： 是否有Mike的信， $s1 = 0$

Brad： 是否有T-boy的信， $s2 = 0$

Mike和T-boy： 树洞是否为空， $s3 = 1$

Mike:	T-boy:	Lincon:	Brad:
write()	write()	P(s1)	P(s2)
P(s3)	P(s3)	getit()	getit()
putit()	putit()	V(s3)	V(s3)
V(s1)	V(s2)	readit()	readit()

同濟大學軟件學院

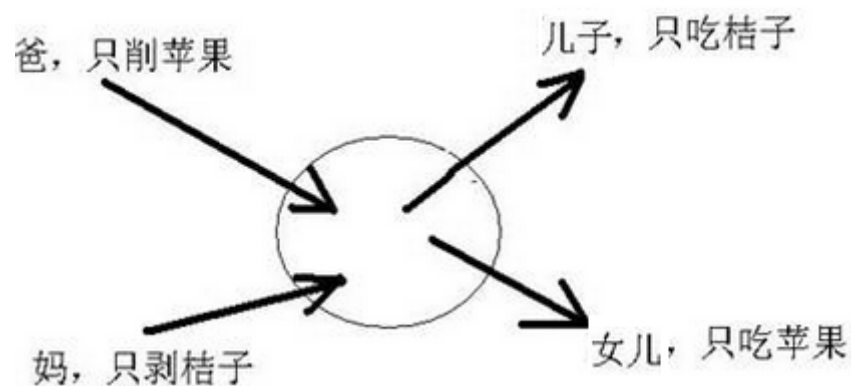


练习



● 吃水果

问题描述：桌上有一只盘子，每次只能放入**五个水果**。爸爸专向盘中放苹果，妈妈专向盘中放桔子，一个女儿专等吃盘中的苹果，一个儿子专等吃盘中的桔子。



需要几个信号量？几个进程？试写出各进程流程。

同濟大學軟件學院

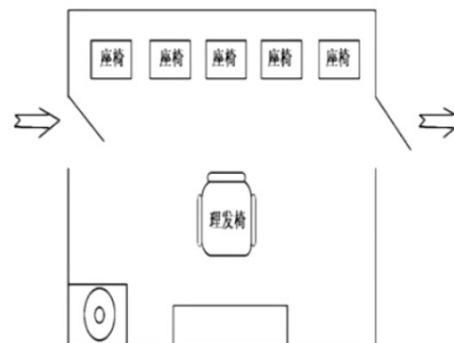


练习



● 理发师

- 理发店由等待间（包含N个等待座位）和理发间（包含一把理发椅）构成
- 无顾客时，理发师睡觉；当顾客进入理发店发现理发师睡觉时，则唤醒理发师
- 请写出模拟理发师和顾客行为的算法



打瞌睡的理发师

需要几个信号量？几个进程？试写出各进程流程。

同濟大學軟件學院



练习



在一个仓库中可以存放 A 和 B 两种产品，要求：

- ①每次只能存入一种产品。
- ②A 产品数量 - B 产品数量 $< M$ ，其中 M 是正整数。
- ③B 产品数量 - A 产品数量 $< N$ ，其中 N 是正整数。

假设仓库的容量是无限的，试用 P, V 操作描述产品 A 和 B 的入库过程。

【解答】 使用信号量 mutex 控制两个进程互斥访问临界资源（仓库），使用同步信号量 Sa 和 Sb（分别代表产品 A 与 B 的还可容纳的数量差，以及产品 B 与 A 的还可容纳的数量差）满足条件 2 和条件 3。代码如下：

```
Semaphore Sa=M-1, Sb=N-1;
Semaphore mutex=1;          //访问仓库的互斥信号量
process_A(){
    while(1){
        P(Sa);
        P(mutex);
        A 产品入库;
        V(mutex);
        V(Sb);
    }
}

process_B(){
    while(1){
        P(Sb);
        P(mutex);
        B 产品入库;
        V(mutex);
        V(Sa);
    }
}
```



小结



- 临界区
- 同步互斥方法
- 信号量机制

